Outcome-Aligned Task Offloading for DNN Inference in Heterogeneous Fog Networks

Jeng-Shyang Pan[1,2], Peng-Yu Duan[1], Li-Fa Liu[1], Qing-Yong Yang[1], Shu-Chuan Chu[3*]

[1]College of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao, 266590, China.
[2]Department of Information Management, Chaoyang University of Technology, Taichung, 41349, Taiwan.
[3]School of Artificial Intelligence, Nanjing University of Information Science and Technology, Nanjing, 210044, China.

*Corresponding author(s). E-mail(s): 100130@nuist.edu.cn;
Contributing authors: jspan@ieee.org; 202483060107@sdust.edu.cn; 202282060085@sdust.edu.cn; yang_qy@sdust.edu.cn;

Abstract

We evaluate whether scoring assignments after feasibility repair improves whole-request DNN placement in a five-node synthetic fog network. Repair-based fitness and genotype–phenotype separation are established techniques; our contribution is a controlled empirical study. A 2x2 experiment across six loads and 30 paired runs per load caps each arm at 10,000 candidate-score calls. With local search disabled, repair-aware ranking improves satisfaction by 8.82–29.83 percentage points across loads (14.99 points on the equal-load mean; 95% CI 14.06–15.92). The inherited five-update heuristic changes no assignments in 1,784,010 updates. Repair-aware scoring performs 10,000 candidate repairs per run versus one final repair for raw ranking, so equal score calls do not imply equal computation. In a separate 500-slot replay, the ALM-labelled implementation averages 46.84% satisfaction at $\boldsymbol{N = 180}$ and 3.85% at $\boldsymbol{N = 300}$, but every method reaches the queue cap in every $\boldsymbol{N = 300}$ run. Both protocols use synthetic workloads; queued tasks do not age toward their latency allowance. The results establish neither queue stability, deployed performance, equal-compute superiority, nor a new ALM principle.

Keywords: Fog computing, DNN inference, task offloading, repair-aware ranking

1 Introduction

Latency-sensitive DNN inference can use fog clusters whose nodes differ in compute speed, memory, and communication quality. The scheduler must choose one node per request while respecting deadlines and resource limits. In the implemented simulator, congestion couples assignment choices through shared compute load and context-switching overhead.

Practical schedulers often generate a complete assignment and then repair it to satisfy memory and load constraints. Search can rank the raw assignment even though only its repaired form is deployed. Consequently, a high-scoring raw genotype need not produce the best repaired schedule. The question is whether evaluating a temporary repaired phenotype improves the delivered service enough to justify its computational cost.

We study this question for whole-request DNN placement in a five-node synthetic fog pool. The genotype stores one node choice per request; repair is used for evaluation without overwriting the genotype. Repair-aware fitness and genotype–phenotype separation are established optimization patterns. The intended contribution is a controlled application study of their benefits, costs, and failure cases in this simulator, not a new repair theory or augmented-Lagrangian method.

The study addresses three questions:

- Does repair-aware ranking improve admitted, timely, resource-feasible service relative to raw ranking under the same metric definition?
- Does the ranking effect persist when the optional local heuristic is disabled, and does enabling that heuristic change discrete placements?
- What computational and modeling limitations restrict interpretation of the observed gains?

The new strict V2 factorial experiment addresses the ranking question and the local-heuristic ablation using exact candidate-score budgets and a local-search-off control. It provides simulator-specific evidence for repair-aware ranking, while showing that the inherited local heuristic changes no discrete assignments. It does not test two evolutionary families or isolate an acceptance guard. Historical experiments are excluded from the current results because their selection logic, polishing policy, or source attestations do not match strict V2. A separate 500-slot retry-state replay is reported as a distinct, heavily overloaded synthetic scenario; it is not used to infer the strict V2 ranking effect.

2 Related Work

2.1 Mobile Edge Computing and Fog Computing Architectures

Mobile Edge Computing (MEC) and Fog Computing place compute and network resources closer to end users, with the aim of supporting latency-sensitive services [1, 2]. Surveys by Mach et al. [3] and Abbas et al. [4] review MEC architectures and computation offloading. Kumari et al. survey task-offloading algorithms and

optimization techniques in fog computing [5], while Zhou et al. review the intersection of edge computing and artificial intelligence [6].

These architectural studies motivate the present work but do not by themselves resolve the control problem addressed here. Once a heterogeneous fog tier is fixed, the main challenge shifts to how a controller allocates model-specific tasks under hard memory limits, unequal compute capacities, fluctuating communication costs, and repeated congestion.

2.2 Computation Offloading Optimization

Task-offloading studies optimize different criteria, including energy consumption, delay, and long-term cost. Examples include energy-aware offloading in heterogeneous networks [7, 8], delay-oriented scheduling and queue-aware policies [9, 10], and binary offloading in wireless-powered MEC [11]. Other work addresses software-defined dense networks, edge intelligence, and task offloading under different system models [6, 12, 13]. Studies also examine IoT-oriented edge computing, security-aware offloading, and user-centric decisions [14–17]. Recent C-RAN/MEC papers consider joint edge-server/service deployment and user-satisfaction-based offloading, respectively [18, 19]. These papers provide context, not like-for-like baselines or a systematic review of repair-aware evolutionary selection; no novelty claim is inferred from an asserted absence in the literature.

The cited studies pursue different objectives, including energy minimization, delay, long-term cost, and user satisfaction, under different system assumptions. They provide context rather than like-for-like baselines for the present simulator. Our repeated-slot replay is therefore analyzed separately from the static ranking experiment and is not used to imply that prior work lacks stateful scheduling methods.

2.3 Deep Learning Inference at the Edge

Edge-side DNN inference introduces system properties that are absent from generic scalar-task models. Neurosurgeon studies partitioned execution between mobile devices and the cloud; Edgent combines device-edge partitioning with early exits; and BottleNet compresses intermediate features for mobile-cloud execution [20–22]. Distributed DNN execution and adaptive DNN surgery further examine how deployment hierarchy and network dynamics affect inference partitioning [23, 24].

In our benchmark, DNN task classes have distinct workload and memory parameters. The present simulator nevertheless treats each request as an indivisible whole-model task; it does not model layer partitioning or adaptive inference, so those mechanisms are outside the evaluated contribution.

2.4 AI-Driven Optimization (DRL)

Deep reinforcement learning (DRL) has been applied to dynamic offloading problems. The original deep Q-network work demonstrated learned control in Atari games rather than MEC scheduling [25]; subsequent studies applied deep reinforcement learning to online or decentralized MEC offloading under changing channel or load conditions

Table 1 Positioning of the present study against representative prior work categories

Work	Decision and objective	Method and reported evidence
Zhang et al. [7]	Energy-efficient MEC offloading and radio allocation	Joint optimization; simulation
Meng et al. [10]	Queue-aware, delay-optimal MEC offloading	MDP-based multi-level water-filling; simulation
Min et al. [28]	Edge choice and offloading rate under energy harvesting	RL/deep-RL; simulated energy, delay, and utility
Ours	**Whole-request fog placement for timely, feasible service**	**Shared repair; exact-score factorial and separate 500-slot retry-state replay**

[26, 27]. These methods provide relevant examples of learned control, but this study does not compare against or validate a DRL scheduler.

The difficulty is operational rather than conceptual. DRL policies often require large training effort, careful reward shaping, and explicit safety handling once hard feasibility constraints are introduced. When infrastructure conditions change quickly, the distinction between "policy quality" and "policy retraining burden" becomes important. This is why snapshot-rerunnable optimization baselines remain relevant even when the evaluation target includes long-horizon behavior.

2.5 Scope and Positioning

Repair-based candidate evaluation, evolutionary search, sigmoid deadline surrogates, softmax relaxation, and augmented-Lagrangian methods are not introduced as new theories here. The proposed question is narrower: how does ranking a repaired schedule change task placement relative to ranking its unrepaired genotype under the same simulator, search representation, and candidate-score budget? Strict V2 uses one reporting evaluator, a fixed score-call cap, paired environments, and a local-search-off arm. Historical results remain outside this evidence because they used different delivery policies or cannot be tied to actual historical source bytes.

The simulator schedules whole DNN requests to one node. It does not implement layer partitioning, early exiting, feature compression, model accuracy adaptation, energy optimization, or learned online control. The replay is a retry queue with slot-local latency allowances, not a validated continuous-time execution or deadline-aging model. The following literature table provides context only; it does not establish that prior work lacks repair-aware selection or that this implementation is state of the art.

3 System Model and Problem Formulation

3.1 Fog Computing Network Architecture

The implemented simulator considers a centralized scheduler and a heterogeneous set of fog nodes $\mathcal{F} = \{1, \ldots, F\}$. The standard benchmark is a one-shot batch placement problem, while the dynamic extension repeats the same placement pipeline over a sequence of decision slots. In both cases, the current codebase models fog execution only:

each task is either assigned to one fog node or later rejected by the feasibility repair stage if no safe placement exists. The resulting formulation is therefore a constrained fog-selection problem rather than a joint local–edge–cloud formulation.

3.2 Task Model and Binary Offloading

Each request corresponds to one DNN inference task. In slot t, the scheduler observes an active task set Ω_t that contains newly arrived requests together with any unresolved backlog carried from the previous slot. Each task $i \in \Omega_t$ is represented by a task type and a deadline,

$$\mathcal{T}_i = \{\tau_i, T_i^{\max}\}, \tag{1}$$

where $\tau_i \in \{1, 2, 3\}$ denotes VGG16, ResNet50, or MobileNetV2, respectively. The detailed communication and computation costs are not generated from a symbolic workload formula inside the optimizer; instead, they are obtained from precomputed matrices parameterized by fixed synthetic profiles. No empirical measurement provenance is supplied with the archived timing table. Let

$$x_{i,f}(t) \in \{0, 1\} \tag{2}$$

denote the assignment decision, with $x_{i,f}(t) = 1$ when task i is mapped to fog node f in slot t. We also define an explicit admission indicator

$$a_i(t) \in \{0, 1\}, \tag{3}$$

where $a_i(t) = 1$ means that task i remains assigned after repair and $a_i(t) = 0$ means that the final repaired schedule rejects the task. Before repair, the optimizer produces exactly one node choice per active task:

$$\sum_{f \in \mathcal{F}} x_{i,f}(t) = 1, \quad \forall i \in \Omega_t. \tag{4}$$

After repair, some tasks may be dropped, which is represented in the evaluation code by an all-zero row and therefore satisfies

$$\sum_{f \in \mathcal{F}} x_{i,f}(t) = a_i(t), \quad \forall i \in \Omega_t. \tag{5}$$

3.3 Profile-Parameterized Synthetic Workload

The five-node pool contains two Nano-like, two Pi-like, and one high-capacity server-like node. These are simulator labels, not measurements of deployed devices. Task types are sampled with probabilities 0.4, 0.3, and 0.3 for VGG16, ResNet50, and MobileNetV2; their nominal latency allowances are 3, 2, and 1 s with uniform multiplicative jitter in $[0.9, 1.1]$. Input plus model weights approximate per-task memory; shared model caching and intermediate activations are not modeled.

The static generator assigns fixed base execution times (Pi-like, Nano-like, server-like), in seconds: MobileNetV2 $(0.150, 0.030, 0.001)$, ResNet50 $(1.200, 0.200, 0.005)$, and

VGG16 $(4.500, 0.800, 0.020)$. These constants have no accompanying raw measurement records, device settings, or sampling protocol. They must therefore be treated as synthetic parameters pending provenance verification. Dividing each constant among six placeholder segments and summing it back does not create empirical trace data or a partitioning decision.

Replay uses a different preprocessing path: $\text{Comp}_{i,f} = W_i/C_f$, with workloads $(15.5, 3.8, 0.6)$ GFLOP for the three task types and numerical node rates $(100, 100, 50, 50, 5000)$. We interpret these rates as effective GFLOP/s for dimensional consistency; the legacy code's GHz comments are not hardware measurements. The static timing table is not reused by replay. Thus differences between static and replay include the compute model as well as backlog carry-over; they cannot isolate the effect of adding temporal state.

3.4 Communication and Transmission Model

The preprocessing code uses a deterministic path-loss Shannon-rate proxy:

$$R_{i,f} = B_f \log_2 \left(1 + 0.1 g_{i,f}/N_0\right), \tag{6}$$

$$g_{i,f} = 10^{-[128.1+37.6 \log_{10}(d_{i,f}/1000)]/10}, \tag{7}$$

$$\text{Comm}_{i,f} = 8 \times 10^6 D_i / R_{i,f}, \tag{8}$$

where $d_{i,f}$ is in meters and floored at 1 m, D_i is input size in MB, and N_0 is the noise power calculated from $-174 + 10\log_{10}(20 \times 10^6)$ dBm. Nodes use fixed bandwidth values of 500 MHz or 10 GHz. The model does not allocate shared airtime or include interference, fading, downlink delay, or bandwidth-dependent noise scaling; it is not a validated OFDMA resource-allocation model. Positions are sampled in a 500×500 m square. The per-user uplink-rate field generated elsewhere is not used in this matrix calculation.

3.5 Computation Model with Static Batch Congestion Penalty

The simulator penalizes overloaded nodes by combining a static batch congestion term with an explicit context-switching overhead. Let

$$L_f(t) = \sum_{i \in \Omega_t} x_{i,f}(t)\, \text{Comp}_{i,f} \tag{9}$$

be the aggregate compute load on node f in slot t, and let $\bar{T}_t = \frac{1}{|\Omega_t|} \sum_{i \in \Omega_t} T_i^{\max}$ be the mean task deadline used as a normalization window. The effective utilization is

$$\rho_f(t) = \frac{L_f(t)}{\bar{T}_t}. \tag{10}$$

The congestion multiplier used in the code is

$$q_f(t) = \begin{cases} 1 + \dfrac{\rho_f(t)}{2(1-\rho_f(t)+\epsilon)}, & \rho_f(t) < 0.99, \\ 10^4, & \rho_f(t) \geq 0.99, \end{cases} \tag{11}$$

where ϵ is a small numerical stabilizer. Let $K_f(t) = \sum_{i \in \Omega_t} x_{i,f}(t)$ denote the number of tasks assigned to node f. The context-switching overhead applied to all tasks on node f is

$$T_f^{ctx}(t) = \alpha_{ctx} K_f(t)^2, \tag{12}$$

with $\alpha_{ctx} = 500\mu s$. Therefore, the end-to-end latency of task i placed on node f is

$$T_i(f,t) = \mathrm{Comm}_{i,f} + \mathrm{Comp}_{i,f}\, q_f(t) + T_f^{ctx}(t). \tag{13}$$

This is the latency model actually used by the evaluation code.

3.6 Resource Feasibility and Repair Semantics

The evaluator marks a node as resource-violating when its assigned task memory exceeds capacity or its normalized load is at least 0.99. Safe Harbor repair (also called G3R in legacy comments) processes tasks in ascending minimum base compute time. It retains the original node if memory fits, normalized load is below 0.95, and the inserted task meets its allowance. Otherwise it chooses the lowest modeled latency among memory-fitting nodes below 0.95, even if the task is late; if no such node exists it returns a zero row. Later insertions can also make earlier tasks late. Repair therefore enforces a memory/load packing rule, not a guarantee that all admitted tasks meet their deadlines.

The earlier V1 pipeline subsequently performed up to three randomized task-order reassignment sweeps. That polishing stage used a load cutoff of 0.999, while the reporting evaluator penalizes loads at 0.99; its checks did not establish global monotonicity of satisfaction. Strict V2 removes this post-selection polishing stage and delivers the common repair output directly. The prior threshold mismatch is therefore not exercised in the strict V2 results and those earlier polished tables are excluded.

3.7 Retry Replay Semantics

Each replay slot adds N new tasks to the algorithm-specific unresolved queue Q_t, producing $|\Omega_t| = N + Q_t$. After evaluation, satisfied tasks leave; all other tasks may be retried, with older rows retained first up to cap $Q^{\mathrm{cap}} = N$. Let C_t be the satisfied count and D_t the queue-cap loss. The implemented accounting is

$$Q_{t+1} = \min\{Q^{\mathrm{cap}}, |\Omega_t| - C_t\}, \tag{14}$$

$$D_t = |\Omega_t| - C_t - Q_{t+1}. \tag{15}$$

Pending task types, user positions, and original latency allowances persist, but elapsed waiting time is not subtracted from those allowances. Partial service is not retained,

node positions are resampled, and task identifiers are renumbered in each active batch. This is retry-state persistence, not tracking physical execution or absolute arrival-to-completion deadlines. No physical slot duration or optimization walltime is charged to service latency.

3.8 Objective and Reported Metrics

For a binary schedule $\mathbf{Y}$, define admission $a_i = \sum_f y_{i,f} \in \{0,1\}$. The reporting evaluator uses

$$US_i(\mathbf{Y}) = \mathbf{1}\{a_i = 1\}\mathbf{1}\{T_i(\mathbf{Y}) \le T_i^{\max}\}\mathbf{1}\{\text{assigned node is resource-feasible}\}. \quad (16)$$

A zero row earns zero satisfaction and counts as a composite failure; its zero algebraic row sum must never be interpreted as successful zero latency. Satisfaction is $100 \sum_i US_i/|\Omega_t|$. Resource violation and deadline/admission failure are reported separately when available; composite violation is exactly 100 minus satisfaction and is not an independent benefit.

Strict V2 retains one node index per genotype and ranks either the raw assignment or its repaired phenotype $\widehat{\mathbf{Y}} = \mathcal{R}(\mathbf{X})$. Comparisons prioritize feasible over infeasible candidates; among feasible candidates they prefer higher satisfaction, and among infeasible candidates they prefer lower resource-violation rate. The same reporting evaluator supplies both values. The delivered assignment is exactly $\mathcal{R}(\mathbf{X}^\star)$; there is no post-selection polishing. Therefore, in the repair-aware arm, the scored phenotype and delivered assignment coincide. The ALM-named local heuristic is tested as an optional inherited component and is not claimed as a novel theory.

Replay exports mean slot satisfaction, total satisfied tasks, backlog sum, peak and final backlog, queue-cap losses, and runtime. Its mean slot score is

$$\overline{US} = \frac{100}{H} \sum_{t=1}^{H} \frac{C_t}{|\Omega_t|}. \quad (17)$$

This is an average of per-attempt ratios with algorithm-dependent active sets, not the fraction of all arrivals meeting an end-to-end deadline. Total completions and the conservation identity

$$HN = \sum_t C_t + \sum_t D_t + Q_{H+1}, \quad Q_1 = 0, \quad (18)$$

provide complementary count-based checks. Backlog peak is denoted Q^{peak}, separately from Q^{cap}. The canonical replay export does not support the previously stated weighted long-horizon utility, P95/P99 latency, or failure-aware post-processing objective; these are removed rather than presented as implemented evidence.

Algorithm 1: Corrected repair-aware evolutionary search; local heuristic is optional

Input: Tasks, nodes, repair operator, population P, generations G
Output: Assignment after the shared final repair
Initialize integer population and evaluate it with the shared score;
for $g = 1$ **to** G **do**
  foreach *genotype* **X do**
    Optionally generate a local proposal using the disclosed heuristic;
    Score parent and proposal using the same evaluator and repair setting;
    If enabled, accept the proposal only under the stated lexicographic rule;
    Preserve the integer genotype for crossover and mutation;
  end
  Update elite; apply tournament selection, crossover, mutation, and elitism;
end
Apply the shared final repair to the selected elite;
Measure satisfaction, resource status, admission, and end-to-end runtime;

4 Repair-Aware Candidate Revision

4.1 Scope and Representation

The integer population, tournament selection, uniform crossover, mutation, and elitism are retained. The correction changes the score, not the application or workload. A temporary repair phenotype is scored without writing it back to the genotype; this is a non-Lamarckian evaluation convention, not a newly proposed evolutionary theory. Any diversity benefit requires direct measurement and is not inferred from that convention.

4.2 Shared Admission-Aware Selection

The strict V2 selector calls `calculate_metrics_v2` for candidate scoring and final reporting, converts percentage satisfaction to a fraction, and uses the evaluator resource-violation field as the feasibility key. Repair-aware candidates are first transformed by $\mathcal{R}$; raw candidates are not repaired before scoring. A final common repair is applied to both selected outputs, and no polishing follows. Rejected tasks receive no satisfaction credit.

4.3 Local Heuristic and Discrete Projection

The archived formal configuration explicitly set one inner update per local-search invocation, with invocation probability 0.25 per individual and generation. The historical solver's default of five updates was overridden. Logits start at $+2$ for the chosen node and -2 otherwise; Adam state resets on each invocation. With one update, each coordinate moves by at most the learning rate 0.1, so an initial logit margin of 4 remains at least 3.8. Argmax is unchanged. Thus the archived formal local proposals cannot alter integer placements; equality of guarded and unguarded outcomes does not establish a useful safety mechanism. The strict V2 factorial experiment uses five inner updates when enabled and includes a matched local-search-off arm. Across 1,784,010 updates,

no integer gene changed. The local heuristic is not part of the defensible primary method; repair-aware ranking is evaluated with it disabled.

The direction computed from soft probabilities is applied directly to logits without the softmax Jacobian, and the multiplier term omits cross-task load derivatives. It is an ALM-inspired heuristic, not the exact gradient of a fully specified augmented Lagrangian. No convergence or new optimization principle is claimed.

4.4 Cost and Fair Comparison

Sorting plus node scanning makes one repair pass $O(N \log N + NF)$. The full shared evaluator includes unsuccessful-task diagnostics that can rescan other tasks for each candidate; its current implementation can cost $O(N^2 + NF)$ per score. Repair-aware population evaluation must include this cost. Final polishing can cost $O(N^2F)$ per sweep because repeated task-wide sums occur inside candidate-node loops.

Strict V2 uses a fixed cap of 10,000 candidate-score calls in every factorial arm. The initial population is scored first, and the remaining budget is consumed by candidate and local-proposal evaluations; a local invocation costs two score calls and contains five inner updates. Every run asserts and records the exact realized count. Repair-aware arms additionally invoke the shared feasibility repair once per candidate score, so equal score calls do not imply equal computation or wall time.

The strict V2 factorial comparison holds the evolutionary engine, initialization protocol, population size, random seeds, and score-call cap fixed while varying ranking semantics and local-search execution. It does not match wall-clock time: repair-aware scoring performs more repair operations. Timing includes solver, final repair, and the reporting evaluator; optimization time is not charged to modeled task latency.

5 Experimental Evaluation

5.1 Versioned protocol and audit scope

The primary result is a separately versioned repair-only factorial experiment (MCA-STRICT-V2-REPAIR-ONLY), not a reinterpretation of the earlier polished tables. It crosses raw versus repaired-phenotype ranking with local-search enabled versus disabled in the same legacy evolutionary engine. For every load $N \in \{120, 150, 180, 200, 250, 300\}$, 30 environment/run pairs are evaluated. All four arms share the generated environment, solver seed, population size 100, guarded acceptance setting, and an exact cap of 10,000 candidate-score calls. Disabled arms retain the same invocation-random-number draw but execute zero local updates. Because enabled invocations consume two score calls, fixed score budgets can produce different generation counts; the design matches score calls, not search trajectories or wall-clock time.

The raw-ranking arms score the unrepaired genotype; the repair-aware arms first apply the same deterministic feasibility repair $\mathcal{R}$ and score the repaired phenotype with the reporting metric. In both cases, final delivery applies $\mathcal{R}$ once and then the same reporting evaluator. No polishing is applied. Thus repair-aware ranking evaluates the object delivered, while raw ranking is the explicit control. The evaluator's 0.99 load threshold defines resource violation for both selection and reporting. The earlier

Table 2 Strict V2 mean satisfaction (%) across 30 paired runs. Final delivery applies the same repair and no polishing.

N	Raw/off	Raw/on	Repair/off	Repair/on
120	60.06	51.78	89.89	89.86
150	31.44	29.42	40.27	40.27
180	27.19	25.80	38.00	38.00
200	26.07	25.03	36.02	36.02
250	17.76	17.75	34.84	34.84
300	18.76	18.71	32.18	32.16

post-selection polisher used a separate 0.999 threshold but is absent from strict V2. Satisfaction is the fraction of tasks both admitted and within their latency allowance. Search comparisons prefer feasible over infeasible candidates, maximize satisfaction among feasible candidates, and minimize resource violation among infeasible candidates; no convergence or global-optimality claim is made.

The completed design contains 720 unique run/load/arm rows. Each row records score calls, evaluator calls, repair calls, local invocations, inner updates, changed integer genes, solver-plus-final-repair time, reporting time, and their end-to-end sum. All arms use 10,000 score calls; resource violations are zero after final repair; environment and solver seeds pair exactly. Historical hash files remain defective and cannot attest earlier experiments. The strict V2 sources have a new actual-byte SHA-256 snapshot; it identifies the current code only and does not retroactively validate old results.

We report per-load paired differences with pointwise paired-t 95% intervals and paired sign-flip tests (20,000 draws, plus-one correction); Holm adjustment is applied across the 24 load-specific factorial contrasts. For the four load-averaged factorial contrasts and the frozen learning comparator, loads are averaged equally within each run before inference, leaving 30 independent run pairs. The intervals are pointwise, not simultaneous. A nonsignificant contrast is not treated as evidence of equivalence.

5.2 Repair-aware ranking under an exact score-call budget

Tables 2 and 3 report the four arm means and paired effects. Repair-aware ranking improves satisfaction in every tested load; differences range from 8.82 to 29.83 percentage points. All six load-specific ranking contrasts have 30/30 positive pairs and Holm-adjusted sign-flip $p = 0.0012$ across 24 factorial contrasts. Averaging six loads equally within each of the 30 runs gives a 14.99-point difference (95% CI [14.06, 15.92]; paired sign-flip $p = 0.00005$, Holm-adjusted $p = 0.00020$ across four aggregate factorial contrasts). This comparison isolates ranking within this implemented search and synthetic simulator, not a universal repair principle or an improvement over all offloading methods.

Table 3 Paired factorial effects by load, in satisfaction percentage points. Intervals are pointwise paired-t 95% CIs; local search is disabled for the ranking contrast and raw ranking is used for the local contrast.

N	Repair-aware − raw [95% CI]	Local-on − off [95% CI]
120	29.83 [24.21, 35.45]	-8.28 [-10.86, -5.70]
150	8.82 [8.03, 9.62]	-2.02 [-2.49, -1.56]
180	10.81 [9.88, 11.75]	-1.39 [-1.81, -0.97]
200	9.95 [8.92, 10.98]	-1.03 [-1.79, -0.28]
250	17.08 [16.66, 17.50]	-0.01 [-0.04, 0.01]
300	13.42 [12.68, 14.16]	-0.04 [-0.14, 0.05]

Table 4 Observed operation counts; cumulative columns sum over 180 evaluations per arm.

Arm	Scores/eval.	Repairs/eval.	Local calls	Inner updates	Changed genes
Raw / local off	10,000	1	0	0	0
Raw / local on	10,000	1	356,802	1,784,010	0
Repair / local off	10,000	10,001	0	0	0
Repair / local on	10,000	10,001	356,802	1,784,010	0

5.3 Local-search ablation and compute accounting

The five-update local heuristic changed zero integer genes across 1,784,010 inner updates. Under raw ranking, enabling it reduced mean satisfaction by 2.13 points (95% CI [-2.61, -1.65]; paired sign-flip $p = 0.00005$, Holm-adjusted $p = 0.00020$ across four aggregate contrasts) because its two candidate-score calls per invocation consumed the fixed search budget without changing the discrete assignment. Under repair-aware ranking, 27 of 30 run-averaged pairs were identical and three were lower with local search enabled; the mean difference was -0.008 points (95% CI [-0.019, 0.002], Holm-adjusted sign-flip $p = 0.247$). This does not establish equivalence, but gives no evidence that the local heuristic improves this discrete objective. The defensible implementation is therefore repair-aware ranking with the local heuristic disabled; the ALM label identifies inherited code only.

Equal candidate-score calls do not equal equal computation: repair-aware scoring performed 10,000 candidate repairs per run, whereas raw ranking performed only the shared final repair. Unified end-to-end timers were recorded, but the formal shards ran concurrently with other workloads. Those wall-clock measurements are retained for audit and are not used to claim relative speed or real-time performance.

5.4 Frozen OfflineGNN comparison

We re-evaluated the previously frozen 24-unit OfflineGNN model on the same 180 load/run environments and random seeds, applying the same final feasibility repair and including inference, repair, and reporting in the timer. The model was not retrained

or reselected for this comparison; its original validation selected the checkpoint under the earlier polishing-enabled pipeline. The result is therefore descriptive for this frozen checkpoint rather than a fully reoptimized learned baseline. Its run-level satisfaction, after equal averaging over six loads, was 24.49 percentage points below RepairLocalOn (95% CI [24.03, 24.96], $n = 30$ run pairs). The comparison describes this particular internal model and hardware-independent simulator; it does not establish superiority over published learning methods. The score-call budget is not applicable to a learned forward pass, so this is not a compute-budget-matched quality comparison.

5.5 500-slot retry-state replay

We completed a separate replay at $N \in \{180, 300\}$ with 20 paired environment runs per scale, 500 consecutive slots, and three implemented methods (Greedy, GAHybrid, and the ALM-labelled implementation). All nine shards contain the same frozen configuration: the GA uses a population of 100 and at most 100 generations; the ALM-labelled implementation uses a population of 100, at most 79 generations and 50 outer iterations, a 0.25 local-search invocation probability, and five inner updates per invocation, with guarded local search and repair-aware fitness enabled. These settings describe the replay implementation and do not isolate the contribution of any one component. The full data contain 60,000 method-slot rows. Independent recomputation from the slot-level exports matched all 120 program-generated run summaries; duplicate keys, missing or incomplete groups, queue-conservation errors, satisfaction-formula errors, and resource-violation rows were zero. This replay uses the $\mathrm{Comp}_{i,f} = W_i/C_f$ preprocessing described above, not the static experiment's latency construction, and is not a factorial test of repair-aware versus raw ranking.

Table 5 reports per-run mean slot satisfaction, cumulative tasks served, queue-cap losses, and the number of runs ending at the queue cap. The mean satisfaction is the equal-weight average of 500 within-slot ratios, then averaged over 20 runs; it is not the fraction of unique arrivals completed by deadline. At $N = 180$, the ALM-labelled method has a higher descriptive mean than either comparator, but run-to-run variability is substantial. At $N = 300$, its paired differences are +0.07 percentage points versus Greedy (95% paired-t CI $[-1.81, 1.95]$) and +1.01 versus GAHybrid (95% CI $[-1.77, 3.79]$); these intervals are descriptive and are not multiplicity-adjusted hypothesis tests. Queue-cap losses range from 26.92% to 44.91% of the 90,000 new arrivals at $N = 180$ and from 92.21% to 94.23% of the 150,000 new arrivals at $N = 300$. Every $N = 300$ run ends at the backlog cap, as do 20/20 Greedy, 20/20 GAHybrid, and 19/20 ALM-labelled runs at $N = 180$. This is evidence of severe overload under the chosen synthetic arrival and capacity settings, not evidence of stable queues or arrival-to-completion deadline performance. Retry allowances do not age while tasks wait, and the reported wall times are not a controlled equal-compute comparison.

The earlier corrected static, guard-ablation, and transfer tables used a different population, polishing stage, and/or unverified historical source manifest. They are excluded from the primary evidence in this version and retained in the audit archive, not relabeled as strict V2 results. The strict V2 factorial experiment answers the ranking and local-heuristic questions directly, but it does not compare every heuristic baseline under its common delivery policy.

Table 5 Recomputed 500-slot retry-state replay results. Satisfaction intervals are 95% t intervals across 20 run-level means; counts are mean per 500-slot run. Terminal backlog-cap frequencies are reported in the text.

N	Method	Slot satisfaction (%; 95% CI)	Tasks served	Queue-cap drops
180	Greedy	28.70 [28.67, 28.74]	51,540	38,280
180	GAHybrid	27.72 [27.59, 27.86]	49,402	40,418
180	ALM-labelled	46.84 [39.05, 54.64]	65,603	24,226
300	Greedy	3.78 [3.23, 4.33]	11,165	138,535
300	GAHybrid	2.84 [0.69, 4.99]	8,351	141,349
300	ALM-labelled	3.85 [2.13, 5.58]	11,386	138,314

6 Discussion

6.1 Supported Interpretation and Attribution

The strict V2 paired experiment shows higher satisfaction for repair-aware than raw ranking in all six tested synthetic loads under an exact candidate-score cap and with local search disabled. This is evidence about the implemented repair and evolutionary search in this simulator. It does not establish a general advantage across arbitrary optimizers, repair operators, hardware, or workload distributions.

Across 1,784,010 local inner updates, no discrete gene changed. With raw ranking, local search reduced satisfaction because its extra score calls consumed the fixed budget without changing assignments. Under repair-aware ranking, 27 of 30 run-averaged pairs were identical and three were slightly lower with local search enabled; the interval includes zero, which is not evidence of equivalence. There is no supported ALM-specific gain, convergence theorem, or safety guarantee. The primary implementation supported by these results disables the inherited local heuristic.

6.2 Novelty and Model Boundaries

Repair-based fitness, integer evolutionary search, and genotype–phenotype separation are established ideas. The defensible contribution is a controlled empirical characterization of repair-aware ranking, local-search behavior, and repair cost in this task-placement model. No new theory or universally novel optimization method is claimed. The experiment does not vary repair-operator design or establish the best repair strength.

The hardware profiles are synthetic and lack device calibration. The wireless proxy omits contention and several physical effects. Static and replay compute preprocessing differ; retry allowances do not age with waiting, and optimization walltime is not charged to task latency. Thus the results do not validate real-device throughput, absolute arrival-to-completion deadlines, queue-stability theory, or deployment feasibility. These are modeling limits that correct statistics cannot remove.

6.3 Audit boundaries and release status

Positive performance estimates do not close the audit. The legacy hash maps do not attest historical source bytes; newly generated hashes identify only the files included in

the current, versioned release. Reproduction records therefore need to bind the exact code snapshot, frozen configurations, result files, and runtime boundary.

The strict V2 score-call counts and local-search-off control are complete and audited; no equal-walltime claim is made because repair-aware scoring performs more repair operations. The full 500-slot replay also passes row-count, key, seed, continuity, queue-flow, and metric checks, but it describes a separate synthetic retry-state protocol and shows severe cap saturation at $N = 300$. The hardware profiles remain synthetic and uncalibrated. The present results do not demonstrate deployment feasibility, deadline aging, stable queues, or general algorithmic superiority. The 95% intervals reported for replay contrasts are descriptive; no multiplicity-adjusted replay hypothesis test is claimed. Reproducibility and visual checks are version-specific and do not repair historical provenance.

6.4 Relationship to a Companion Study

A companion manuscript studies migration-guarded reconfiguration under persistent hotspots. Its migration-budget question differs from the present candidate-ranking question. A SHA-256 comparison of the available versioned artifacts found 20 identical code files, including shared simulator, hardware-profile, and comparator components, but no byte-identical data files across the two reproducibility packages. The present manuscript contains no embedded figures. This file-level check does not exclude derived numerical overlap; the authors should compare the reported result tables and disclose the companion submission to the editor.

7 Conclusion

This study examines repair-aware ranking for whole-request DNN placement while preserving unrepaired integer genotypes. A versioned exact-score-budget factorial experiment shows higher satisfaction for repaired-phenotype ranking than raw ranking across six synthetic loads, including when the inherited local heuristic is disabled. The local heuristic changes no discrete assignment across 1,784,010 inner updates; its repair-aware effect is small and statistically unresolved. These results support a bounded simulator-specific ranking observation, not a new augmented-Lagrangian theory.

The separate 500-slot replay is complete and exposes severe queue-cap saturation under the larger synthetic load; it does not establish stable service or a general ALM advantage. Historical score defects and defective historical hashes prevent treating the old archive as validated evidence; the current versioned outputs do not retroactively repair that provenance. Synthetic hardware calibration and real-device validation remain outside the evidence. The submission package must include the exact current manuscript, code, data, and audit artifacts, with visual and hash checks recorded for that version.

A Implementation Notes

A.1 Local Heuristic Parameters

The archived formal configuration overrides the core default with one update per invocation. The corrected candidate uses five inner updates per invocation. For $N \geq 100$, the wrapper sets learning rate 0.1, initial penalty 0.01, penalty growth 1.05, sigmoid slope 0.8, and internal deadline scaling 0.85. Reporting uses original deadlines. These are heuristic implementation settings and require sensitivity analysis before interpretation as broadly transferable choices.

A.2 Direction, Projection, and Logged Quantity

Softmax forms $p_{i,f} = \exp(z_{i,f}) / \sum_{j=1}^{F} \exp(z_{i,j})$. The local direction combines a sigmoid latency term, clipped load sensitivity, a context-count term, and a deadline multiplier term. The row-sum penalty is zero on the exact softmax simplex. The direction is clipped to $[-5, 5]$ and applied directly to logits through reset Adam state, without the softmax Jacobian. The clipped local load expression is not the evaluator's piecewise congestion function. Accordingly, no equation purporting to be an exact global ALM objective or derivative is asserted. The logged variable called `loss` is mean positive deadline excess under scaled deadlines, not a complete augmented-Lagrangian loss.

The archived one-update setting could not close the initial chosen-versus-other logit margin of 4. In strict V2, the optional heuristic instead performs five inner updates per invocation, but across 1,784,010 updates it still changes no discrete assignment. The factorial local-search-off control is complete; the evidence therefore does not support attributing the repair-aware ranking effect to this heuristic or presenting it as a validated ALM contribution.

A.3 Audit Reproduction and Validation Scope

The audit supplement preserves the defective historical hash maps and a separate corrected candidate. The historical maps are not valid source-byte attestations. A repaired hashing routine records current snapshots separately and cannot reconstruct historical provenance. Scoring tests cover all-rejected, partially admitted, fully admitted, memory-overloaded, load-overloaded, and invalid multi-assignment cases. End-to-end diagnostic runs are labeled separately from formal evidence. Successful arithmetic checks of archived tables or final resource rates do not certify the correctness of historical selection or empirical provenance.

Declarations

Funding

The authors received no specific funding for this work.

Data availability

The dataset entitled "Outcome-Aligned Fog Placement: Strict V2 and 500-Slot Replay" contains the 720 run-level Strict V2 factorial records, the complete 60,000-row slot-level replay, 120 replay run summaries, and the associated analysis outputs. The same versioned repository folder contains the MATLAB source, frozen replay configuration, recomputation scripts, validation reports, and SHA-256 manifests. All experiments use synthetic task and hardware profiles; no measured device traces are included.

Code availability

The versioned source and data release is available at:
`GitHub reproducibility package (version 2026-10-16).`

Author contributions statement

J.-S.P. conceived the study, P.-Y.D. and L.-F.L. conducted the experiments, and Q.-Y.Y. and S.-C.C. analysed the results. All authors reviewed the manuscript.

Additional information

Competing interests

The authors declare no competing interests.

Ethics approval and consent to participate

Not applicable to simulator-only experiments.

Consent for publication

Not applicable.

References

[1] Mao, Y., You, C., Zhang, J., Huang, K., Letaief, K.B.: A survey on mobile edge computing: The communication perspective. IEEE Communications Surveys & Tutorials **19**(4), 2322–2358 (2017) https://doi.org/10.1109/COMST.2017.2745201

[2] Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. IEEE Internet of Things Journal **3**(5), 637–646 (2016) https://doi.org/10.1109/JIOT.2016.2579198

[3] Mach, P., Becvar, Z.: Mobile edge computing: A survey on architecture and computation offloading. IEEE Communications Surveys & Tutorials **19**(3), 1628–1656 (2017) https://doi.org/10.1109/COMST.2017.2682318

[4] Abbas, N., Zhang, Y., Taherkordi, A., Skeie, T.: Mobile edge computing: A survey. IEEE Internet of Things Journal **5**(1), 450–465 (2018) https://doi.org/10.1109/JIOT.2017.2750180

[5] Kumari, N., Yadav, A., Jana, P.K.: Task offloading in fog computing: A survey of algorithms and optimization techniques. Computer Networks **214**, 109137 (2022) https://doi.org/10.1016/j.comnet.2022.109137

[6] Zhou, Z., Chen, X., Li, E., Zeng, L., Luo, K., Zhang, J.: Edge intelligence: Paving the last mile of artificial intelligence with edge computing. Proceedings of the IEEE **107**(8), 1738–1762 (2019) https://doi.org/10.1109/JPROC.2019.2918951

[7] Zhang, K., Mao, Y., Leng, S., Zhao, Q., Li, L., Peng, X., Pan, L., Maharjan, S., Zhang, Y.: Energy-efficient offloading for mobile edge computing in 5g heterogeneous networks. IEEE Access **4**, 5896–5907 (2016) https://doi.org/10.1109/ACCESS.2016.2597169

[8] You, C., Huang, K., Chae, H., Kim, B.-H.: Energy-efficient resource allocation for mobile-edge computation offloading. IEEE Transactions on Wireless Communications **16**(3), 1397–1411 (2017) https://doi.org/10.1109/TWC.2016.2633522

[9] Liu, J., Mao, Y., Zhang, J., Letaief, K.B.: Delay-optimal computation task scheduling for mobile-edge computing systems. In: 2016 IEEE International Symposium on Information Theory (ISIT), pp. 1451–1455 (2016). https://doi.org/10.1109/ISIT.2016.7541539

[10] Meng, X., Wang, W., Wang, Y., Lau, V.K.N., Zhang, Z.: Closed-form delay-optimal computation offloading in mobile edge computing systems. IEEE Transactions on Wireless Communications **18**(10), 4653–4667 (2019) https://doi.org/10.1109/TWC.2019.2926465

[11] Bi, S., Zhang, Y.J.: Computation rate maximization for wireless powered mobile-edge computing with binary computation offloading. IEEE Transactions on Wireless Communications **17**(6), 4177–4190 (2018) https://doi.org/10.1109/TWC.2018.2821664

[12] Chen, M., Hao, Y.: Task offloading for mobile edge computing in software defined ultra-dense network. IEEE Journal on Selected Areas in Communications **36**(3), 587–597 (2018) https://doi.org/10.1109/JSAC.2018.2815360

[13] Deng, S., Zhao, H., Fang, W., Yin, J., Dustdar, S., Zomaya, A.Y.: Edge intelligence: The confluence of edge computing and artificial intelligence. IEEE Internet of Things Journal **7**(8), 7457–7469 (2020) https://doi.org/10.1109/JIOT.2020.2984887

[14] Sun, X., Ansari, N.: Edgeiot: Mobile edge computing for the internet of things. IEEE Communications Magazine **54**(12), 22–29 (2016) https://doi.org/10.1109/MCOM.2016.1600492CM

[15] Premsankar, G., Di Francesco, M., Taleb, T.: Edge computing for the internet of things: A case study. IEEE Internet of Things Journal **5**(2), 1275–1284 (2018) https://doi.org/10.1109/JIOT.2018.2805263

[16] Samy, A., Elgendy, I.A., Yu, H., Zhang, W., Zhang, H.: Secure task offloading

in blockchain-enabled mobile edge computing with deep reinforcement learning. IEEE Transactions on Network and Service Management **19**(4), 4872–4887 (2022) https://doi.org/10.1109/TNSM.2022.3190493

[17] Deng, X., Sun, Z., Li, D., Luo, J., Wan, S.: User-centric computation offloading for edge computing. IEEE Internet of Things Journal **8**(16), 12559–12568 (2021) https://doi.org/10.1109/JIOT.2021.3057694

[18] Pan, J.-S., Yang, Q.-Y., Chu, S.-C., Chao, H.-C.: A joint edge server and service deployment method in c-ran with multi-layer mec for multi-communities. IEEE Internet of Things Journal (2024) https://doi.org/10.1109/JIOT.2024.3471817

[19] Yang, Q.-Y., Chu, S.-C., Hu, C.-C., Kong, L., Pan, J.-S.: A task offloading method based on user satisfaction in c-ran with mobile edge computing. IEEE Transactions on Mobile Computing **23**(4), 3452–3465 (2024) https://doi.org/10.1109/TMC.2023.3275580

[20] Kang, Y., Hauswald, J., Gao, C., Rovinski, A., Mudge, T., Mars, J., Tang, L.: Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. In: Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 615–629 (2017). https://doi.org/10.1145/3037697.3037698

[21] Li, E., Zhou, Z., Chen, X.: Edge intelligence: On-demand deep learning model co-inference with device-edge synergy. In: Proceedings of the 2018 Workshop on Mobile Edge Communications, pp. 31–36 (2018). https://doi.org/10.1145/3229556.3229562

[22] Eshratifar, A.E., Esmaili, A., Pedram, M.: Bottlenet: A deep learning architecture for intelligent mobile cloud computing services. In: 2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED), pp. 1–6 (2019). https://doi.org/10.1109/ISLPED.2019.8824955

[23] Teerapittayanon, S., McDanel, B., Kung, H.T.: Distributed deep neural networks over the cloud, the edge and end devices. In: 2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS), pp. 328–339 (2017). https://doi.org/10.1109/ICDCS.2017.226

[24] Hu, C., Bao, W., Wang, D., Liu, F.: Dynamic adaptive dnn surgery for inference acceleration on the edge. In: IEEE INFOCOM 2019 - IEEE Conference on Computer Communications, pp. 1423–1431 (2019). https://doi.org/10.1109/INFOCOM.2019.8737614

[25] Mnih, V., Kavukcuoglu, K., Silver, D., *et al.*: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015) https://doi.org/10.1038/nature14236

[26] Huang, L., Bi, S., Zhang, Y.-J.A.: Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks. IEEE Transactions on Mobile Computing **19**(11), 2581–2593 (2020) https://doi.org/10.1109/TMC.2019.2928811

[27] Tang, M., Wong, V.W.S.: Deep reinforcement learning for task offloading in mobile edge computing systems. IEEE Transactions on Mobile Computing **21**(6), 1985–1997 (2022) https://doi.org/10.1109/TMC.2020.3036871

[28] Min, M., Xiao, L., Chen, Y., Cheng, P., Wu, D., Zhuang, W.: Learning-based computation offloading for iot devices with energy harvesting. IEEE Transactions on Vehicular Technology **68**(2), 1930–1941 (2019) https://doi.org/10.1109/TVT.2018.2890685